

NAME: S.PADMAJA

Date: 19/06/2026

REG NO:192424159

31. Write a Python program to create a stacked bar plot with error bars.

Aim

To create a stacked bar plot with error bars using Matplotlib. The women's bars are stacked on top of the men's bars using the bottom parameter.

Algorithm

1. Import the required libraries (numpy and matplotlib.pyplot).
2. Define the mean values and standard deviations for men and women.
3. Create positions for the bars using NumPy.
4. Plot the men's bars with error bars.
5. Plot the women's bars on top of the men's bars using the bottom parameter.
6. Add labels, title, and legend.
7. Display the stacked bar chart.

Code:

```
import numpy as np
import matplotlib.pyplot as plt

# Sample data
men_means = (22, 30, 35, 35, 26)
women_means = (25, 32, 30, 35, 29)

men_std = (4, 3, 4, 1, 5)
women_std = (3, 5, 2, 3, 3)

# Position of bars
ind = np.arange(len(men_means))
width = 0.5
```

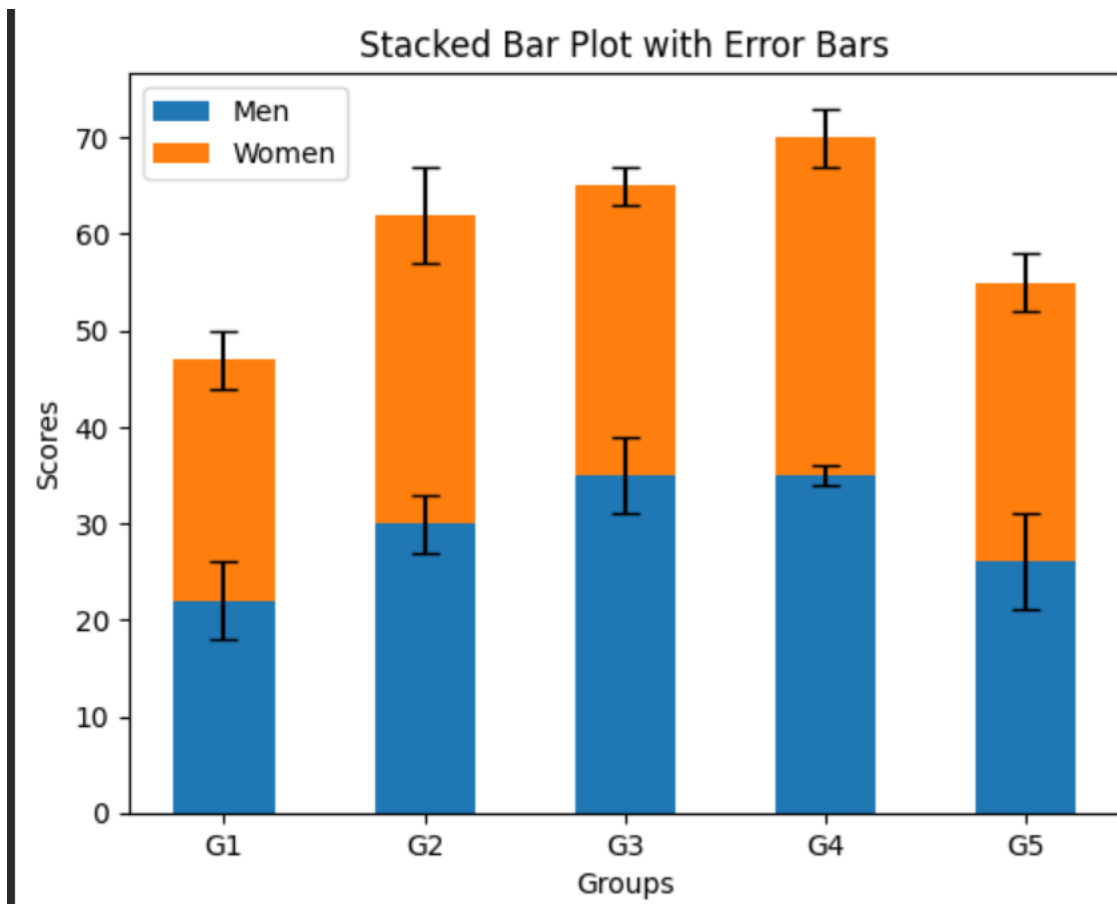
```
# Plot men's bars
p1 = plt.bar(ind, men_means, width,
             yerr=men_std,
             capsize=5,
             label='Men')

# Plot women's bars stacked on men's bars
p2 = plt.bar(ind, women_means, width,
             bottom=men_means,
             yerr=women_std,
             capsize=5,
             label='Women')

# Labels and title
plt.ylabel('Scores')
plt.xlabel('Groups')
plt.title('Stacked Bar Plot with Error Bars')
plt.xticks(ind, ('G1', 'G2', 'G3', 'G4', 'G5'))
plt.legend()

plt.show()
```

Output:



Result:

Thus, a stacked bar plot with error bars was successfully created using Matplotlib, where the women's bars are stacked on top of the men's bars using the bottom parameter.

32. Write a Python program to draw a scatter graph taking a random distribution in X and Y and plotted against each other.

Aim

To create a scatter graph using random values for X and Y and plot them against each other using Matplotlib.

Algorithm

1. Import the required libraries (numpy and matplotlib.pyplot).
2. Generate random values for X and Y using NumPy.
3. Create a scatter plot using plt.scatter().
4. Add title and axis labels.
5. Display the graph.

Code:

```
import numpy as np
import matplotlib.pyplot as plt

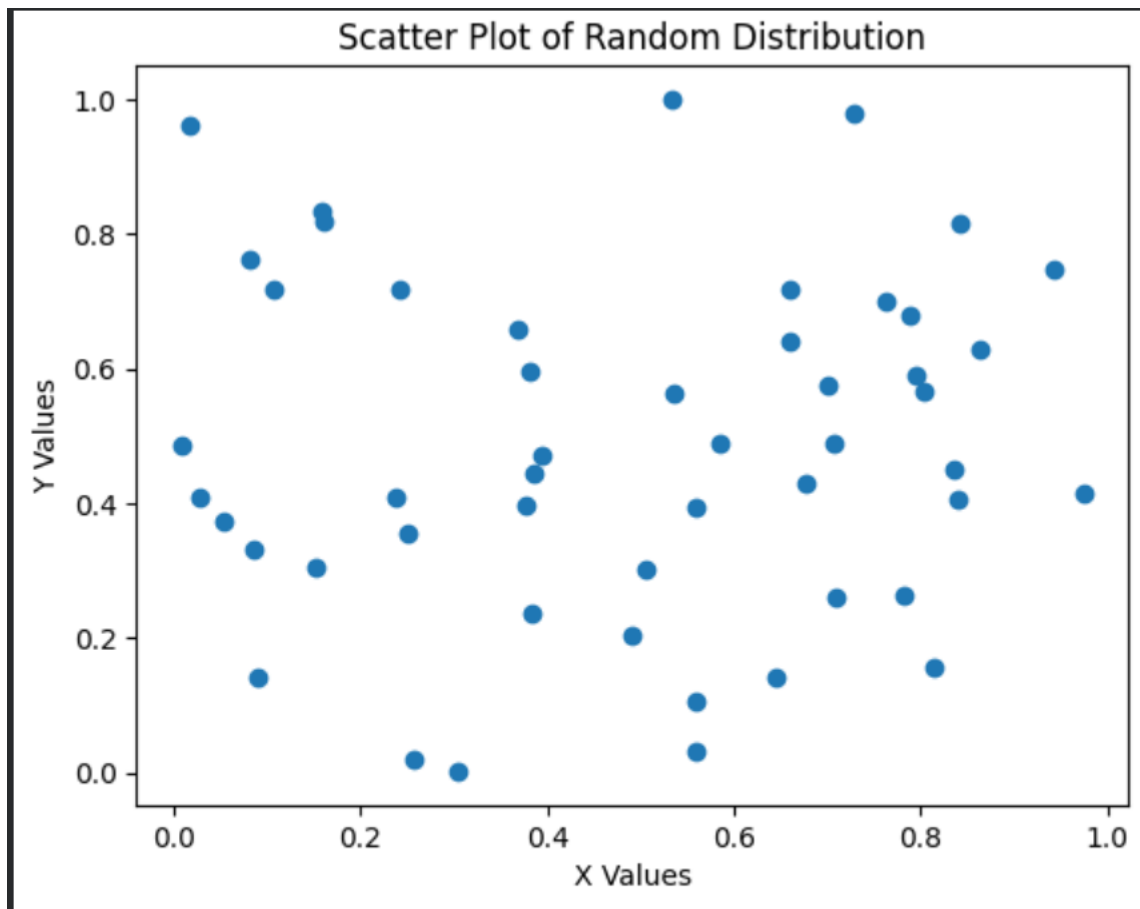
# Generate random data
x = np.random.rand(50)
y = np.random.rand(50)

# Create scatter plot
plt.scatter(x, y)

# Labels and title
plt.xlabel('X Values')
plt.ylabel('Y Values')
plt.title('Scatter Plot of Random Distribution')

# Display graph
plt.show()
```

Output:



Result:

Thus, a scatter graph was successfully created using randomly generated X and Y values and plotted using Matplotlib's `scatter()` function.

33. Write a Python program to draw a scatter plot with empty circles taking a random distribution in X and Y and plotted against each other.

Aim

To draw a scatter plot with empty circles using random distributions of X and Y values and plot them against each other.

Algorithm

1. Import the required libraries: matplotlib.pyplot and numpy.
2. Generate random values for X and Y using NumPy.
3. Create a scatter plot using plt.scatter().
4. Set the marker face color to 'none' to create empty circles.
5. Add title and axis labels.
6. Display the plot.

Code:

```
import matplotlib.pyplot as plt
import numpy as np

# Generate random data
x = np.random.rand(50)
y = np.random.rand(50)

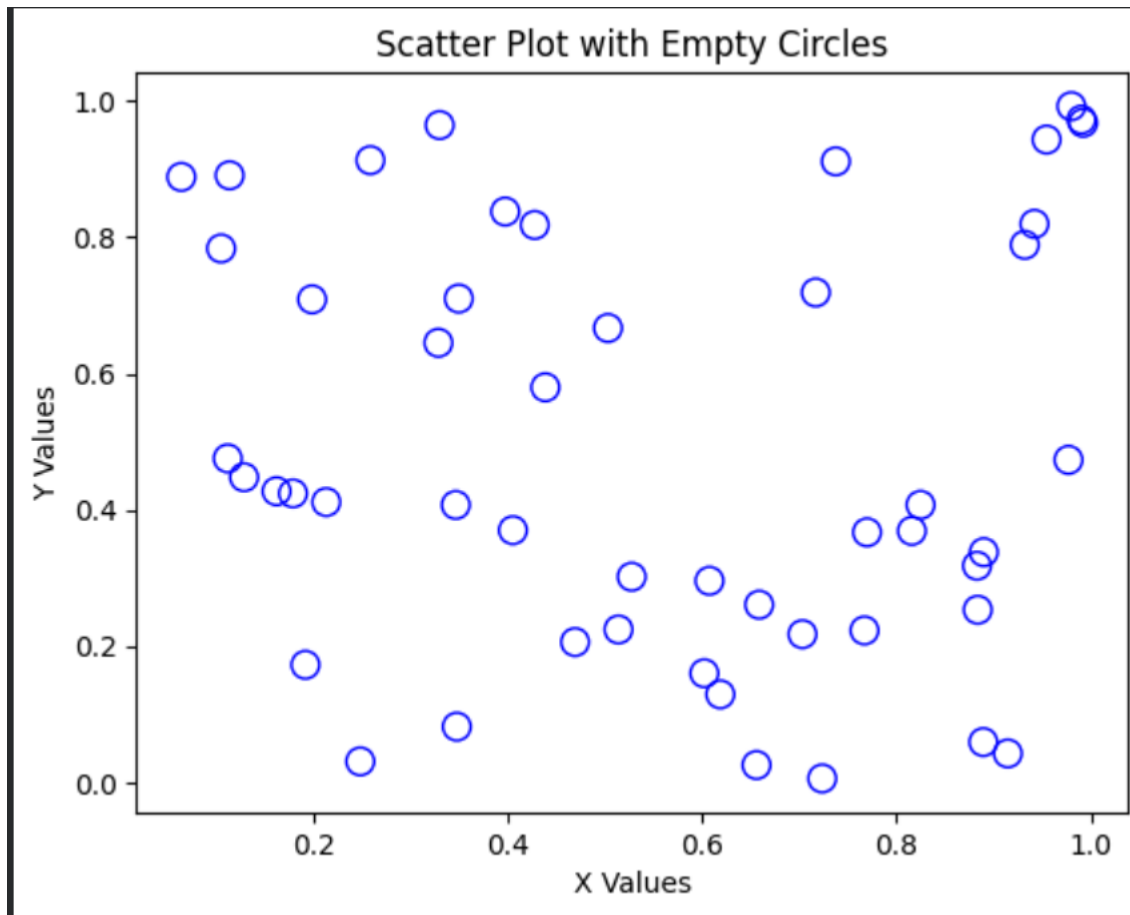
# Create scatter plot with empty circles
plt.scatter(x, y, s=100, facecolors='none', edgecolors='blue')

# Add title and labels
plt.title("Scatter Plot with Empty Circles")
plt.xlabel("X Values")
plt.ylabel("Y Values")

# Display the plot
```

```
plt.show()
```

Output:



Result

Thus, a Python program to draw a scatter plot with empty circles using random distributions in X and Y was successfully executed.

34. Write a Python program to draw a scatter plot using random distributions to generate balls of different sizes.

Aim

To create a scatter plot using random distributions for X and Y coordinates and display points with different ball sizes.

Procedure

1. Import the required libraries (matplotlib and numpy).
2. Generate random values for X and Y coordinates.
3. Generate random values for the sizes of the balls.
4. Create a scatter plot using the generated data.
5. Display the plot with labels and a title.

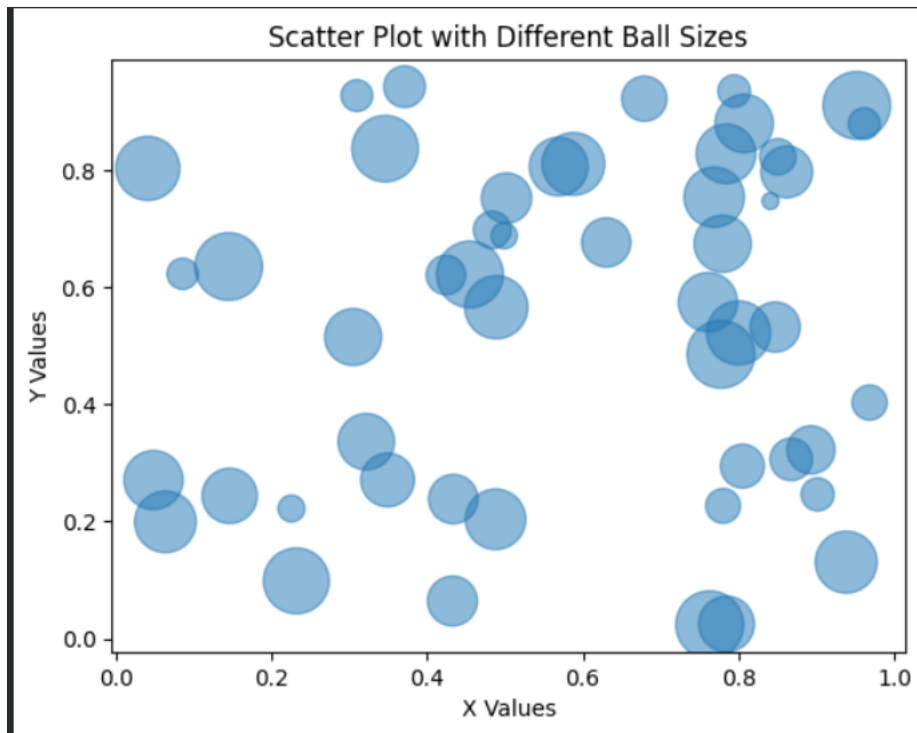
Code:

```
import numpy as np
import matplotlib.pyplot as plt

x = np.random.rand(50)
y = np.random.rand(50)
sizes = np.random.randint(50, 1000, 50)

plt.scatter(x, y, s=sizes, alpha=0.5)
plt.title("Scatter Plot with Different Ball Sizes")
plt.xlabel("X Values")
plt.ylabel("Y Values")
plt.show()
```


Output:



Result:

Thus, a scatter plot using random distributions with balls of different sizes was created and displayed successfully.

35. Write a Python program to draw a scatter plot comparing two subject marks of Mathematics and Science. Use marks of 10 students.

Aim

To draw a scatter plot comparing the marks of 10 students in Mathematics and Science.

Procedure

- 1. Import the matplotlib.pyplot library.**
- 2. Create lists for Mathematics marks, Science marks, and student mark ranges.**
- 3. Use the scatter() function to plot the data.**
- 4. Add a title and labels for the axes.**
- 5. Display the graph using show().**

Code:

```
import matplotlib.pyplot as plt
```

```
math_marks = [88, 92, 80, 89, 100, 80, 60, 100, 80, 34]
```

```
science_marks = [35, 79, 79, 48, 100, 88, 32, 45, 20, 30]
```

```
marks_range = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
```

```
plt.scatter(math_marks, science_marks)
```

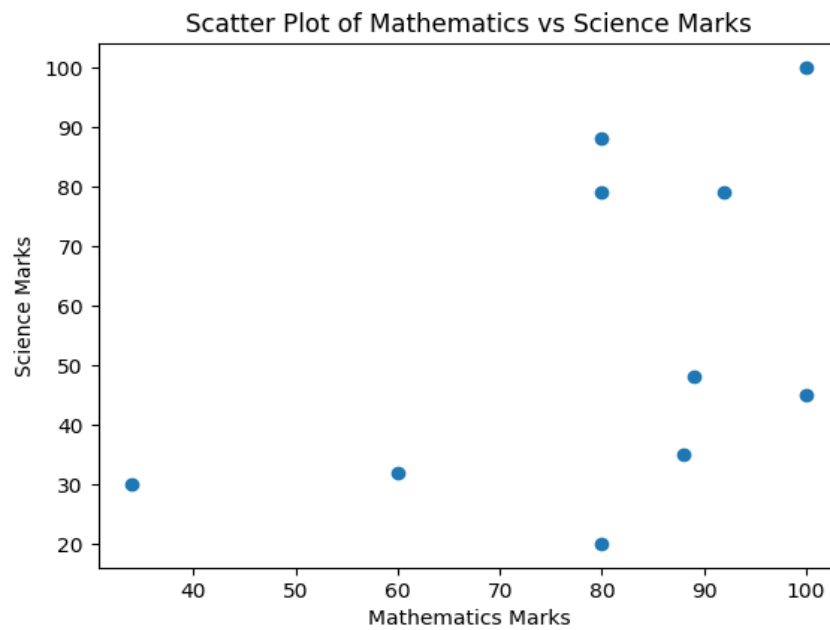
```
plt.title("Scatter Plot of Mathematics vs Science Marks")
```

```
plt.xlabel("Mathematics Marks")
```

```
plt.ylabel("Science Marks")
```

```
plt.show()
```

Output:



Result

The scatter plot comparing Mathematics and Science marks of 10 students was successfully plotted.

36. Write a Python program to draw a scatter plot for three different groups comparing weights and heights.

Aim

To draw a scatter plot for three different groups comparing weights and heights.

Procedure

1. Import the matplotlib.pyplot library.
2. Create separate height and weight data for three groups.
3. Use the scatter() function to plot each group.
4. Add labels, title, and legend.
5. Display the scatter plot.

Code:

```
import matplotlib.pyplot as plt
```

```
height_group1 = [150, 155, 160, 165, 170]
```

```
weight_group1 = [50, 55, 60, 65, 70]
```

```
height_group2 = [152, 158, 163, 168, 172]
```

```
weight_group2 = [52, 57, 62, 67, 72]
```

```
height_group3 = [148, 153, 159, 164, 169]
```

```
weight_group3 = [48, 53, 58, 63, 68]
```

```
plt.scatter(height_group1, weight_group1, label='Group 1')
```

```
plt.scatter(height_group2, weight_group2, label='Group 2')
```

```
plt.scatter(height_group3, weight_group3, label='Group 3')
```

```
plt.title('Height vs Weight Comparison')
```

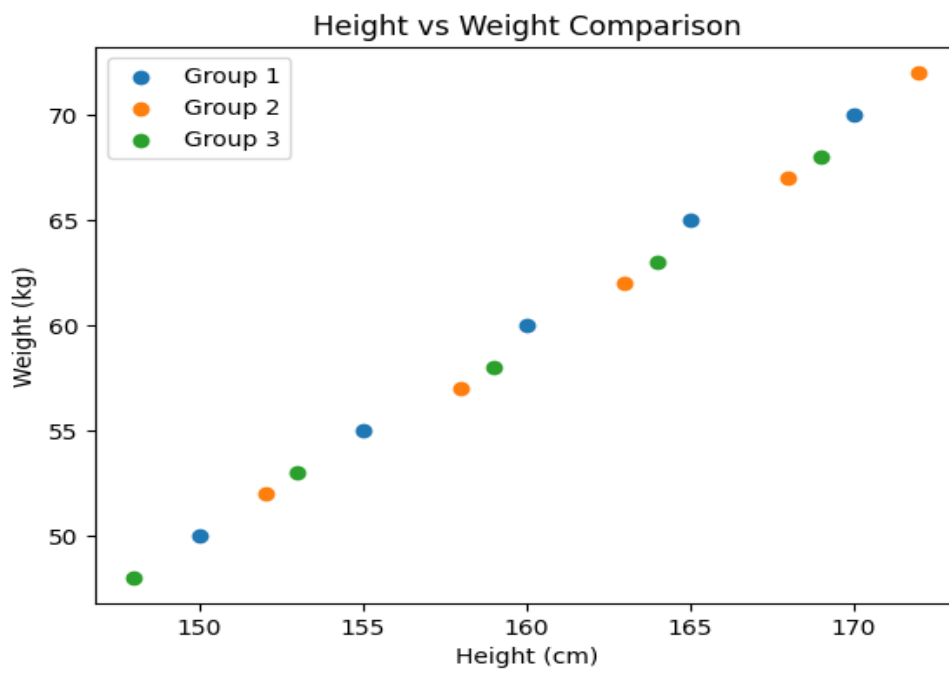
```
plt.xlabel('Height (cm)')
```

```
plt.ylabel('Weight (kg)')
```

```
plt.legend()
```

```
plt.show()
```

Output:



37. Write a Pandas program to create a dataframe from a dictionary**Aim**

To create a Pandas DataFrame from a dictionary and display it.

Procedure

1. Import the Pandas library.
2. Create a dictionary containing the given data.
3. Convert the dictionary into a DataFrame using `pd.DataFrame()`.
4. Display the DataFrame using `print()`.

Code:

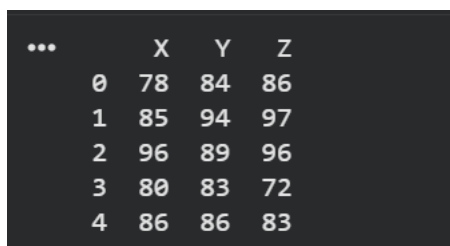
```
import pandas as pd
```

```
data = {  
    'X': [78, 85, 96, 80, 86],  
    'Y': [84, 94, 89, 83, 86],  
    'Z': [86, 97, 96, 72, 83]  
}
```

```
df = pd.DataFrame(data)
```

```
print(df)
```

Output:



...	X	Y	Z
0	78	84	86
1	85	94	97
2	96	89	96
3	80	83	72
4	86	86	83

Result

The DataFrame was successfully created from the given dictionary and displayed.

38. Write a Pandas program to create and display a DataFrame from a specified dictionary data which has the index labels.

Aim

To create and display a Pandas DataFrame from a specified dictionary with custom index labels.

Procedure

1. Import the Pandas and NumPy libraries.
2. Create a dictionary containing the given data.
3. Create a list of index labels.
4. Use pd.DataFrame() to create the DataFrame with the specified index labels.
5. Display the DataFrame using print().

Code:

```
import pandas as pd
```

```
import numpy as np
```

```
exam_data = {  
    'name': ['Anastasia', 'Dima', 'Katherine', 'James', 'Emily',  
            'Michael', 'Matthew', 'Laura', 'Kevin', 'Jonas'],  
    'score': [12.5, 9, 16.5, np.nan, 9, 20, 14.5, np.nan, 8, 19],  
    'attempts': [1, 3, 2, 3, 2, 3, 1, 1, 2, 1],  
    'qualify': ['yes', 'no', 'yes', 'no', 'no',  
              'yes', 'yes', 'no', 'no', 'yes']  
}
```

```
labels = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j']
```

```
df = pd.DataFrame(exam_data, index=labels)
```

```
print(df)
```

Output:

```
..
```

	name	score	attempts	qualify
a	Anastasia	12.5	1	yes
b	Dima	9.0	3	no
c	Katherine	16.5	2	yes
d	James	NaN	3	no
e	Emily	9.0	2	no
f	Michael	20.0	3	yes
g	Matthew	14.5	1	yes
h	Laura	NaN	1	no
i	Kevin	8.0	2	no
j	Jonas	19.0	1	yes

Result

The DataFrame was successfully created from the given dictionary with the specified index labels and displayed.

39. Write a Pandas program to get the first 3 rows of a given DataFrame.

Sample Python dictionary data and list labels:

Aim

To create a Pandas DataFrame from the given dictionary and display the first 3 rows of the DataFrame.

Procedure

1. Import the Pandas and NumPy libraries.
2. Create a dictionary containing student data.
3. Create a list of labels to use as the DataFrame index.
4. Convert the dictionary into a DataFrame.
5. Use the head(3) function to display the first three rows.
6. Print the result.

Code:

```
import pandas as pd
```

```
import numpy as np
```

```
# Dictionary data
```

```
exam_data = {  
    'name': ['Anastasia', 'Dima', 'Katherine', 'James', 'Emily',  
            'Michael', 'Matthew', 'Laura', 'Kevin', 'Jonas'],  
    'score': [12.5, 9, 16.5, np.nan, 9, 20, 14.5, np.nan, 8, 19],  
    'attempts': [1, 3, 2, 3, 2, 3, 1, 1, 2, 1],  
    'qualify': ['yes', 'no', 'yes', 'no', 'no',  
              'yes', 'yes', 'no', 'no', 'yes']  
}
```

```
# Index labels

labels = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j']

# Create DataFrame

df = pd.DataFrame(exam_data, index=labels)

# Display first 3 rows

print(df.head(3))
```

Output:

```
..      name  score  attempts  qualify
a  Anastasia   12.5         1     yes
b      Dima    9.0         3     no
c  Katherine   16.5         2     yes
```

Result

The first three rows of the DataFrame were successfully displayed using the head(3) method.

40. Write a Pandas program to select the 'name' and 'score' columns

Aim

To create a Pandas DataFrame from the given dictionary and select only the 'name' and 'score' columns.

Procedure

1. Import the Pandas and NumPy libraries.
2. Create the dictionary containing student information.
3. Create the DataFrame using the given labels as index.
4. Select the 'name' and 'score' columns using column indexing.
5. Display the selected columns.

Code:

```
import pandas as pd
```

```
import numpy as np
```

```
# Dictionary data
```

```
exam_data = {  
    'name': ['Anastasia', 'Dima', 'Katherine', 'James', 'Emily',  
            'Michael', 'Matthew', 'Laura', 'Kevin', 'Jonas'],  
    'score': [12.5, 9, 16.5, np.nan, 9, 20, 14.5, np.nan, 8, 19],  
    'attempts': [1, 3, 2, 3, 2, 3, 1, 1, 2, 1],  
    'qualify': ['yes', 'no', 'yes', 'no', 'no',  
              'yes', 'yes', 'no', 'no', 'yes']  
}
```

```
# Labels
```

```
labels = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j']
```

```
# Create DataFrame

df = pd.DataFrame(exam_data, index=labels)


# Select 'name' and 'score' columns

result = df[['name', 'score']]


# Display result

print(result)
```

Output:

```
***
```

	name	score
a	Anastasia	12.5
b	Dima	9.0
c	Katherine	16.5
d	James	NaN
e	Emily	9.0
f	Michael	20.0
g	Matthew	14.5
h	Laura	NaN
i	Kevin	8.0
j	Jonas	19.0

Result

The 'name' and 'score' columns were successfully selected and displayed from the DataFrame.